

MANMATHAN S
7376232CB130
DEPARTMENT OF
COMPUTER SCIENCE AND BUSINESS SYSTEMS

SOFTWARE REQUIREMENTS SPECIFICATION

HOSTEL MANAGEMENT PORTAL

TABLE OF CONTENTS

1. INTRODUCTION

- 1.1 Purpose
- 1.2 Scope of the System
- 1.3 Definitions, Acronyms & Abbreviations
- 1.4 References

2. GENERAL DESCRIPTION

- 2.1 Product Perspective
- 2.2 User Class and Characteristics
- 2.3 Design and Implementation Constraints
- 2.4 Assumptions and Dependencies

3. FUNCTIONAL REQUIREMENTS

- 3.1 Admin Functional Requirements
- 3.2 Instructor Functional Requirements
- 3.3 Student Functional Requirements
- 3.4 System Functional Requirements
- 3.5 Grievance & Workflow Requirements

4. EXTERNAL INTERFACE REQUIREMENTS

- 4.1 User Interfaces
- 4.2 Software Interfaces
- 4.3 Communication Interfaces
- 4.4 Hardware Interfaces

5. NON-FUNCTIONAL REQUIREMENTS

- 5.1 Performance Requirements
- 5.2 Security Requirements
- 5.3 Reliability Requirements
- 5.4 Availability
- 5.5 Maintainability
- 5.6 Scalability

6. SYSTEM MODELS

- 6.1 System Architecture
- 6.2 ER Diagram (Database Schema)

7. CONSTRAINTS, RISK AND FUTURE ENHANCEMENTS

- 7.1 Constraints
- 7.2 Risks
- 7.3 Future Enhancements

8. PROJECT SCHEDULE AND BUDGET

- 8.1 Project Schedule
- 8.2 Budget Estimation

9. APPENDICES

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for the **Hostel Management Portal (HMP)**. This document serves as the blueprint for the design and development of a centralized web platform tailored for educational institutions and residential hostels.

The system addresses the need for a unified platform that combines traditional hostel administration—such as room allocation and student record management—with digitized services like grievance redressal, mess management, and gate-pass automation. It is intended for use by Chief Wardens, Block Wardens, and Student Residents to ensure the facility meets the institution's operational and safety goals.

1.2 Scope of The System

The Hostel Management Portal is a full-stack web application hosted on a local or cloud server environment. It facilitates the entire lifecycle of a student's stay, from room application to vacating the premises.

In-Scope Functionalities:

- **User Management:** Role-Based Access Control (RBAC) for Admins, Instructors, and Students using JWT authentication.
- **Infrastructure Management:** Admins can configure Hostel Blocks, Floors, and Rooms, including setting capacity limits (e.g., 2-sharing, 4-sharing).
- **Room Allocation Engine:** A logic-based system to assign rooms to students based on course, year of study, and availability, preventing double-booking.
- **Grievance Redressal:** A ticketing system where students can report facility issues (Electrical, Plumbing, Hygiene) and track their resolution status in real-time.
- **Leave & Out-Pass System:** A workflow for students to apply for leave or gate passes, which Wardens can digitally approve or reject.
- **Mess Management:** Display of weekly food menus and collection of student feedback/attendance for billing purposes.
- **Analytics Dashboard:** Visualization of room occupancy rates, pending complaints, and student attendance statistics stored in MySQL.

Out-of-Scope:

- Live video surveillance or CCTV integration.
- Direct Payment Gateway integration (MVP tracks fee status but does not process transactions).
- Native mobile applications (The web interface is responsive for mobile browsers).

1.3 DEFINITIONS, ACRONYMS & ABBREVIATIONS

Acronym	Description
SRS	Software Requirements Specification
HMP	Hostel Management Portal
RBAC	Role-Based Access Control
JWT	JSON Web Token (Stateless authentication)
ORM	Object-Relational Mapping (Hibernate/JPA)
Ticket	A digital record of a specific complaint or service request
Warden	The staff member responsible for the discipline and management of a hostel block
Out-Pass	Digital authorization for a student to leave the hostel premises
Verdict	The final decision on a request (e.g., Approved, Rejected)

1.4 REFERENCES

- Spring Boot Documentation: <https://docs.spring.io/spring boot/docs/current/reference/html/>
- React Documentation: <https://react.dev/reference/react>
- MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.

2. GENERAL DESCRIPTION

2.1 Product Perspective

The system is a self-contained web application utilizing a **Three-Tier Architecture**:

1. **Frontend (Presentation Layer):** Built with **React.js** and **Tailwind CSS**, operating as a Single Page Application (SPA).
2. **Backend (Application Layer):** Built with **Java Spring Boot**, handling business logic, authentication (**Spring Security**), and the workflow engine.
3. **Database (Persistence Layer):** MySQL is used for storing relational data (users, rooms, complaints), while the server file system stores uploaded documents (medical certificates/ID proofs).

2.2 User Class and Characteristics

- **Administrator (Chief Warden):** System owner with full access. Manages warden accounts, defines hostel blocks, sets fee structures, and oversees overall hostel health.
- **Warden (Staff): Operational managers.** They handle room allocation for specific blocks, approve/reject leave requests, update complaint statuses, and view floor-level analytics.
- **Student (Resident):** End-users. End-users. They view room details, apply for leave, raise complaints, and check mess menus.

2.3 Design and Implementation Constraints

- **Tech Stack:** The system must be developed using React.js (Frontend), Spring Boot (Backend), and MySQL (Database).

- **Security:** Passwords must be hashed using **BCrypt**. API access requires valid **JWT Bearer tokens**.
- **Data Integrity:** The Room Allocation module must use database transactions to prevent "race conditions" where two students are assigned the same bed simultaneously.
- **Storage:** Large files (images/PDFs) are stored on the local file system; the database only stores the file paths.

2.4 Assumptions and Dependencies

- **Assumption:** The host server has the Java Development Kit (JDK 17+) and MySQL Server installed.
- **Assumption:** Users have modern web browsers (Chrome, Edge, Firefox) with JavaScript enabled.
- **Dependency:** The system relies on a running MySQL service on port 3306.

3. FUNCTIONAL REQUIREMENTS

3.1 Admin Functional Requirements

- **User Management:**
 - The system shall allow the Admin to view, search, and manage all users.
 - Only Admins can create or promote users to "Warden" roles.
 - Admins shall be able to reset passwords for locked accounts.
- **System Configuration:**
 - The Admin shall manage global settings (e.g., Academic Year start/end dates) and post system-wide announcements/circulars.
 - Admins can configure "Block Types" (Boys/Girls) to restrict allocations.
- **Audit Logs:**

- The system shall maintain immutable logs of critical actions (e.g., student eviction, room capacity changes) for security and accountability.

3.2 Warden Functional Requirements

- **Room Allocation:**
 - Wardens shall be able to assign a student to a specific room.
 - The system must validate that the room has available capacity before confirming allocation.
 - Wardens can swap rooms between two students upon request.
- **Complaint Management:**
 - Wardens shall view a dashboard of pending complaints for their assigned block.
 - Wardens shall update the status of complaints (e.g., OPEN -> IN_PROGRESS -> RESOLVED) and add remarks.
- **Leave Approvals:**
 - Wardens shall receive notifications for new leave/out-pass requests.
 - Wardens can Approve or Reject requests; rejections must require a comment explaining the reason.

3.3 Student Functional Requirements

- **Profile & Enrollment:**
 - Students shall be able to view their profile, including allocated Room Number, Block Name, and Roommate details.
- **Service Interface:**
 - **Complaints:** Students can raise tickets for maintenance issues by selecting a category (Electrical, Civil, Mess) and attaching a description.
 - **Leave Application:** Students can apply for leave by selecting dates and uploading supporting documents (e.g., medical certificate).
- **Mess Information:**
 - Students shall be able to view the daily breakfast, lunch, and dinner menus.
 - Students can mark themselves "Absent" for mess rebate (if applicable).

3.4 System Functional Requirements

- **Authentication:** The system will use Spring Security to authenticate users and issue JWTs.
- **File Handling:** The system will validate file types (PDF/JPG) and sizes (<5MB) upon upload and serve them via secure HTTP.
- **Auto-Validation:** The backend will automatically reject room allocation requests if the student gender does not match the block type (e.g., Male student in Female block).

3.5 Grievance & Workflow Requirements

- **Submission Queue:** Complaint submissions shall be queued to prevent data loss during high traffic.
- **Status Workflow:** The system shall strictly enforce state transitions (e.g., a complaint cannot go from RESOLVED back to OPEN without a "Re-open" trigger).
- **Supported Categories:** ELECTRICAL, PLUMBING, CARPENTRY, MESS, DISCIPLINARY, INTERNET/WIFI.

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interfaces

- **Frontend:** React.js + Tailwind CSS.
- **Navigation:**
 - **Admin/Warden:** Responsive Sidebar navigation containing links to Dashboard, Rooms, Users, and Complaints.
 - **Student:** Top-bar navigation for Profile, Mess, and Services.
- **Dashboard:** Card-based layout displaying key metrics (e.g., "5 Pending Complaints," "10 Empty Beds").

4.2 Software Interfaces

- **Database:** MySQL 8.0 communicating via Spring Data JPA (Hibernate).

- **OS:** Cross-platform compatibility (Windows/Linux/macOS) for the host server.
- **Web Server:** Embedded Apache Tomcat (Spring Boot default).

4.3 Communication Interfaces

- **Protocol:** RESTful API over HTTP/HTTPS.
- **Data Format:** JSON for all API requests and responses.

4.4 Hardware Interfaces

- **Server:** Minimum 4GB RAM (8GB recommended for video handling), Quad-Core CPU.
- **Storage:** SSD required for database performance; sufficient HDD space for document storage.

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

- **Latency:** API response time must be < 200ms for standard requests (e.g., fetching room list).
- **Throughput:** Support for 100+ concurrent users (e.g., students checking leave status simultaneously).
- **Reporting:** Generating the monthly hostel attendance report should complete within 5 seconds.

5.2 Security Requirements

- **Encryption:** Passwords must be hashed using BCrypt; all data in transit must be encrypted via HTTPS.
- **Access Control:** Strict API endpoint protection ensuring Students cannot access Warden/Admin routes (e.g., /api/admin/**).
- **Input Validation:** All inputs must be sanitized to prevent SQL Injection and Cross-Site Scripting (XSS) attacks.

5.3 Reliability Requirements

- **Data Integrity:** MySQL transactions (ACID) must be used to ensure room allocation data is

never corrupted, even during server crashes.

- **Error Handling:** The system shall display graceful error messages (e.g., "Room Full") to the user instead of raw system stack traces.

5.4 Availability

- The system should aim for 99.9% uptime during academic sessions.
- Scheduled maintenance (e.g., DB backups) should occur during off-peak hours (12:00 AM - 4:00 AM).

5.5 Maintainability

- **Code Structure:** Backend follows the **Controller -> Service -> Repository** pattern for separation of concerns.
- **Configuration:** Database URLs, credentials, and file paths must be externalized in application properties.

5.6 Scalability

- **Statelessness:** The backend is stateless (JWT-based), allowing for horizontal scaling by adding more server instances behind a load balancer if the user base grows.

6. SYSTEM MODELS

6.1 System Architecture

The system follows a monolithic architecture with a clear separation between frontend and backend.

- **USER (Browser):** React.js Client.
- **PRESENTATION LAYER:** Handles UI rendering and user inputs.
- **APPLICATION LAYER (Spring Boot):**
 - **Security Layer:** Validates JWTs and Permissions.
 - **Controller Layer:** REST API Endpoints (/api/auth, /api/rooms).
 - **Service Layer:** Business Logic (RoomService, ComplaintService).

- **PERSISTENCE LAYER: Spring Data JPA.**
- **DATA LAYER: MySQL Database & File Storage System.**

6.2 Er Diagram (Database Schema)

The database schema consists of the following key entities:

- **USERS TABLE**
 - user_id (PK, INT): Unique Identifier.
 - full_name (VARCHAR): Name of the user.
 - role (ENUM): ADMIN, WARDEN, STUDENT.
 - password_hash (VARCHAR): Encrypted password.
- **BLOCKS TABLE**
 - block_id (PK, INT).
 - name (VARCHAR): e.g., "Cauvery Block".
 - type (ENUM): BOYS, GIRLS.
- **ROOMS TABLE**
 - room_id (PK, INT).
 - block_id (FK): Links to Blocks table.
 - room_number (VARCHAR): e.g., "101".
 - capacity (INT): Max students.
 - current_occupancy (INT).
- **ALLOCATIONS TABLE**
 - allocation_id (PK, INT).
 - student_id (FK): Links to Users table.
 - room_id (FK): Links to Rooms table.
 - allocated_at (DATETIME).
- **COMPLAINTS TABLE**
 - complaint_id (PK, INT).
 - student_id (FK).
 - category (VARCHAR).
 - description (TEXT).
 - status (ENUM): OPEN, RESOLVED.
- **LEAVE_REQUESTS TABLE**
 - leave_id (PK, INT).

- student_id (FK).
- start_date (DATE).
- end_date (DATE).
- status (ENUM): PENDING, APPROVED, REJECTED.

7. CONSTRAINTS, RISK AND FUTURE ENHANCEMENTS

7.1 Constraints

- **Resource Intensity:** Generating complex reports (e.g., "All Maintenance History for 2025") may consume significant CPU resources.
- **Storage Limits:** Local server storage limits the number of high-resolution document uploads.

7.2 Risks

- **Data Loss:** Server hardware failure could result in the loss of allocation data.
 - *Mitigation:* Regular automated SQL dumps and file backups.
- **Unauthorized Access:** Students guessing Warden passwords.
 - *Mitigation:* Enforce strong password policies and strict RBAC checks.

7.3 Future Enhancements

- **Biometric Integration:** Integration with hardware fingerprint scanners for automated attendance tracking.
- **Mobile App:** Development of a native Android/iOS app for push notifications regarding complaints and circulars.
- **Cloud Migration:** Moving document storage to AWS S3 for better scalability

8. PROJECT SCHEDULE AND BUDGET

8.1 Project Schedule

Phase	Duration	Key Activities
Planning	Week 1 (Day 1-3)	Gather requirements, finalize SRS.
Design	Week 1 (Day 4-7)	DB Schema design, UI prototyping (Figma).
Core Dev	Weeks 2	Setup Spring Boot/React, User Auth, Room CRUD.
Workflow Dev	Weeks 3	Implement Complaint Logic, Leave Workflow, Allocation logic.
Integration	Weeks 4	Connect Frontend/Backend, Testing.
Deployment	Week 4	UAT, Server setup, Final Launch.

8.2 Budget Estimation

- **Software:** Open-source (Free) - Java, MySQL, VS Code.
- **Infrastructure:** Local Server (Existing Institution Hardware) or VPS (\$40/mo).
- **Development Cost:** Internal Student/Faculty effort (Academic/Zero direct cost)

9. APPENDICES

- **Appendix A:** API Endpoint Documentation (Swagger/OpenAPI).
- **Appendix B:** Setup Guide for Wardens (How to allocate rooms).
- **Appendix C:** Glossary of Terms (LMS, RBAC, JWT, etc.).